



FAKULTI TEKNOLOGI DAN KEJURUTERAAN
ELEKTRONIK DAN KOMPUTER
UNIVERSITI TEKNIKAL MALAYSIA MELAKA

TECHNOLOGY SYSTEM OPTIMIZATION II

BERL 3114

SEMESTER 1

SESI 2025/2026

MINI PROJECT: SMART TRAFFIC LIGHT SYSTEM

NO.	STUDENTS' NAME	MATRIC. NO.
1.	MUHAMMAD AZRUL BIN REDZUAN	B122310626
2.	NURSERI BINTI WASRI	B122310552
3.	NUR FAQIHAH BINTI OTHMAN	B122310255
4	FAISAL HAQIEM BIN FIRDAUS	B122310031
PROGRAMME	3 BERL	
SECTION / GROUP	SMART TRAFFIC LIGHT	
DATE	16/1/2026	
NAME OF INSTRUCTOR(S)	MUHAMMAD FAREQ BIN IBRAHIM	
EXAMINER'S COMMENT(S)		TOTAL MARKS

Content	Page
1.0 Introduction	1
2.0 System Operation	2
2.1 Flow Chart	
2.2 Blok Diagram	3
3.0 Circuit Diagram	5
3.1 GPIO Table	6
4.0 Coding	7
5.0 Result Of Prototyping	14
5.1 Hardware	
5.2 Software	17
6.0 Costs Of The Project	21
7.0 Conclusion	22
8.0 Link for presentation	23

1.0 INTRODUCTION

Traffic light systems play a crucial role in controlling traffic flow and ensuring road safety at intersections. In addition to managing vehicle movement, modern traffic light systems also consider **pedestrian safety**, especially in urban areas where pedestrian crossings are common. An effective traffic control system must balance both vehicle efficiency and pedestrian protection.

This project focuses on the design and development of a **traffic light prototype using an ESP32 microcontroller**, incorporating three LED indicators: **Red, Yellow, and Green**. The system is designed to simulate the basic operation of a real-world traffic light, including the **concept of pedestrian waiting and crossing time**, although no physical pedestrian button is implemented in this prototype.

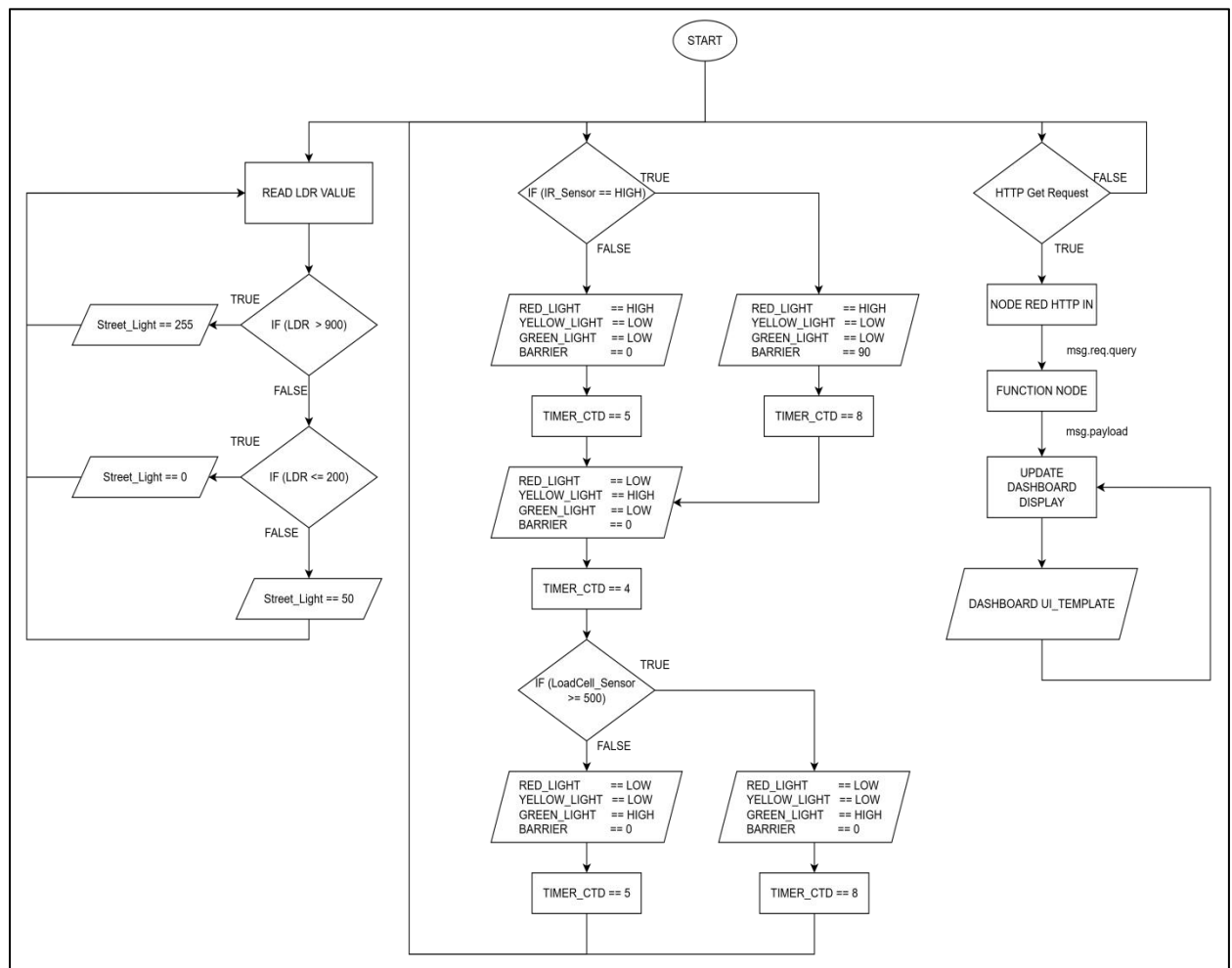
The ESP32 is used as the main controller due to its **high processing capability, low power consumption, and flexibility for future upgrades**, such as pedestrian push buttons or sensor-based detection. The programmed timing sequence ensures that vehicles are stopped during the red signal, allowed to move during the green signal, and warned during the yellow signal, which indirectly provides a safe crossing period for pedestrians.

This project aims to help students understand the fundamental principles of **embedded systems, digital input/output control, and timing-based logic** using a practical and relatable application. By developing this traffic light prototype, the project demonstrates how a simple system can be expanded in the future to include **pedestrian crossing features**, making it suitable for smart traffic and smart city applications.

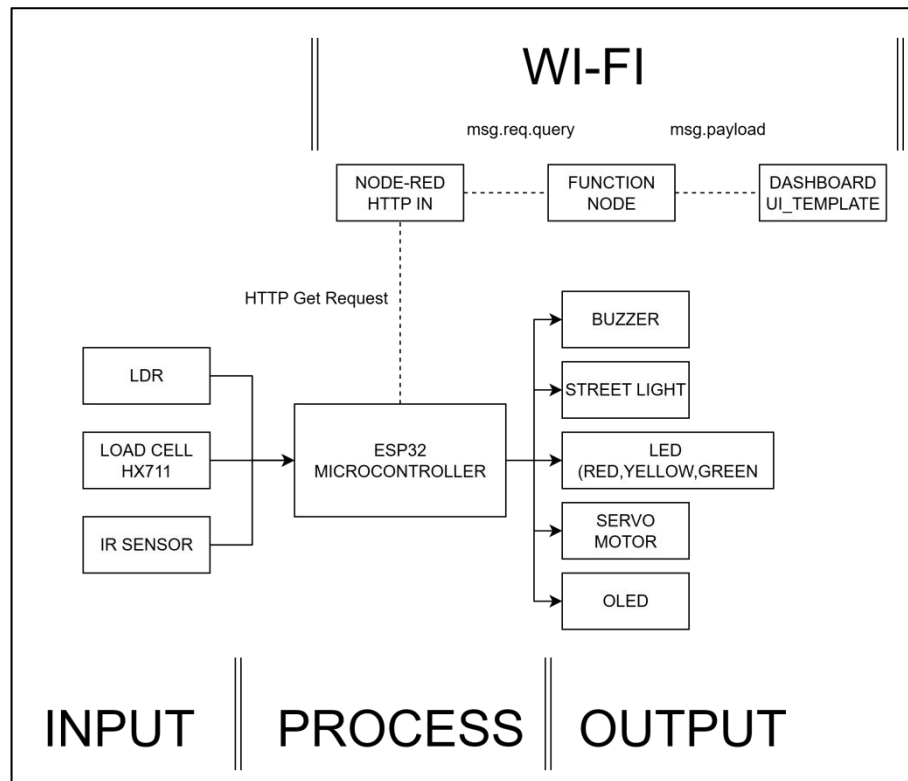
2.0 SYSTEM OPERATION

- The system is controlled by an **ESP32 microcontroller**.
- The traffic light operates using a **fixed timing sequence**.
- The **Red LED** turns ON to indicate vehicles must stop.
- During the red light phase, **pedestrians are assumed to cross the road safely**.
- After the red light duration, the **Green LED** turns ON to allow vehicle movement.
- While the green light is ON, **pedestrian crossing is not allowed**.
- The **Yellow LED** turns ON as a warning signal for vehicles to prepare to stop.
- The yellow light operates for a **short transition period**.
- After the yellow phase, the system returns to the **Red LED**.
- The entire traffic light sequence **repeats continuously**.
- All LED operations are controlled using **ESP32 GPIO pins** and **software delay functions**.
- The system can be **upgraded in the future** to include pedestrian buttons, sensors, or countdown indicators.

2.1 Flow Chart



2.2 Block Diagram



This system is designed to monitor environmental conditions and vehicle presence, and to control multiple output devices automatically using an **ESP32 microcontroller** as the main processing unit.

i. Input Block

- The input block consists of several sensors that collect real-time data from the environment:
- The **LDR sensor** is used to detect ambient light intensity to determine day or night conditions.
- The **IR sensor** is used to detect the presence or movement of objects or vehicles.
- The **Load Cell sensor** measures weight to detect vehicle presence based on applied load.

All sensor data is sent to the ESP32 for further processing.

ii. Process Block

The **ESP32 microcontroller** acts as the central controller of the system. It processes data from all input sensors, executes programmed control logic, and determines the appropriate actions for the output devices. The ESP32 also manages network communication via Wi-Fi.

iii. Wi-Fi Communication Block

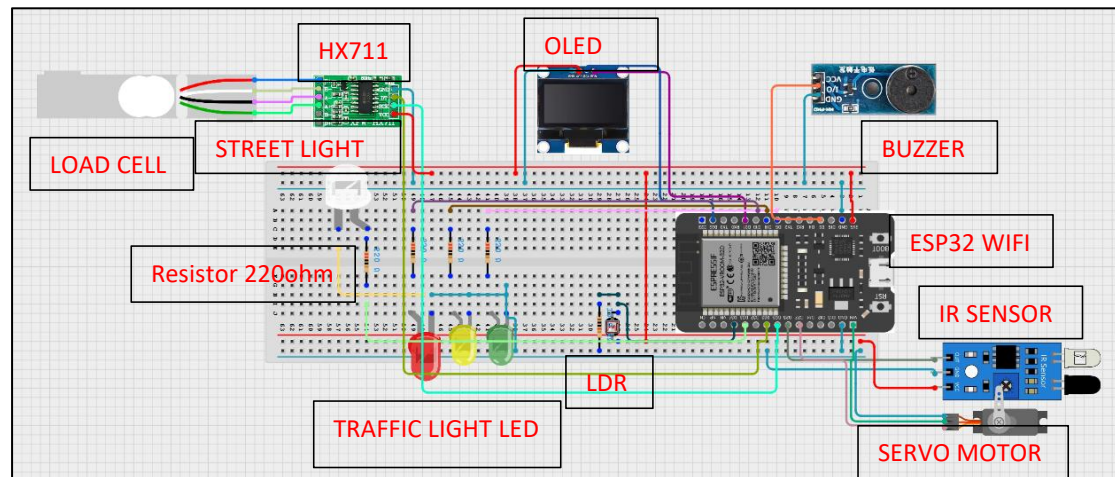
The ESP32 communicates with **Node-RED** through a Wi-Fi connection using the **HTTP protocol**. The data flow is as follows:

- The **HTTP In node** receives data from the ESP32.
- The **Function node** processes the incoming data and applies additional logic if required.
- The **Node-RED Dashboard** displays real-time sensor readings and system status for monitoring and visualization.

iv. Output Block

- The output block consists of various devices controlled by the ESP32:
- The **Servo motor** operates mechanical movement such as opening or closing a barrier.
- The **OLED display** shows system status, sensor values, and traffic information.
- The **Buzzer** provides an audible alert when specific conditions are detected.
- **Traffic light LEDs (Red, Yellow, Green)** indicate traffic flow status.
- The **Street light LED** is automatically turned ON or OFF based on ambient light conditions.

3.0 CIRCUIT DIAGRAM



The circuit diagram shows the hardware connections of the system designed using the Cirkuit website, where all components are interfaced with an ESP32 microcontroller as the main controller. The LDR sensor is connected to an analog pin to detect ambient light levels, the IR sensor is connected to a digital input to detect object or vehicle presence, and the load cell sensor is connected through an HX711 module to measure weight accurately. Output devices including the servo motor, traffic light LEDs, street light LED, buzzer, and OLED display are connected to the ESP32 via appropriate digital, PWM, and I²C pins. All components share a common ground and are powered according to their voltage requirements to ensure stable and reliable system operation.

3.1 GPIO Table

COMPONENT	PIN	GPIO	DESCRIPTION
OUTPUT			
Red LED, Yellow LED, Green LED	VCC	3.3V	As a traffic light sequence
	GPIO	14,12,13	
	GND	GND	
OLED Display	VCC	3.3V	To display countdown of red light and sequence traffic light
	GND	GND	
	SDA	21	
	SCK	22	
Street Light (PWM)	Anode	25	emitted light according to brightness
	Cathode	GND	
Servo Motor	VCC	5V	As a barrier to block a traffic to let pedestrian crossing
	GPIO	27	
	GND	GND	
Buzzer	VCC	3.3V	make a 1Hz sound before barrier up
	GPIO	17	
	GND	GND	
INPUT			
LDR	Anode	3.3V	To control street light brightness
	Cathode	34	
Load Cell sensor	VCC	3.3V	Detect vehicle presence
	GND	GND	
	DT	32	
	SCK	33	
IR Sensor	VCC	3.3V	Detect pedestrian to trigger traffic light
	GPIO	2	
	GND	GND	

4.0 CODING

Library:

```
#include <WiFi.h>
#include <HttpClient.h>
#include <ESP32Servo.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "HX711.h"
```

WIFI Configuration:

```
const char* ssid = "HONOR 400 Pro";
const char* password = "aaaaaaaa";
const char* serverUrl =
```

OLED Setup:

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT,
```

Traffic Light, IR sensor & Servo Setup:

```
int ledRed = 14;
int ledYellow = 12;
int ledGreen = 13;
int irSensor = 2;
Servo barrier;
```

Traffic Light state machine:

```
enum TrafficState { RED, GREEN, GREEN_BLINK, YELLOW };
TrafficState currentState = RED;
```

Timer Variable:

```
unsigned long stateMillis = 0;
unsigned long redTime = 5000;
unsigned long greenTime = 5000;
```

Pedestrian & Barrier Logic:

```
bool pedestrianWaiting = false;
bool pedestrianReported = false;
bool barrierUp = false;
```

Load Cell and HX711 Setup:

```
HX711 scale;
int loadCellDT = 32;
int loadCellSCK = 33;
float weightThreshold = 50.0;
bool vehicleDetected = false;
```

Street Light setup:

```
int ldrPin = 34;
int streetLedPin = 25;
int streetBrightness = 0;
```

Timers:

```
unsigned long oledMillis = 0;
unsigned long sendMillis = 0;
unsigned long loadMillis = 0;
unsigned long ldrMillis = 0;
unsigned long blinkMillis = 0;
bool blinkState = false;
int blinkCount = 0;
```

Void setup:

```
void setup() {  
  pinMode(ledRed, OUTPUT);  
  pinMode(ledYellow, OUTPUT);  
  pinMode(ledGreen, OUTPUT);  
  pinMode(irSensor, INPUT_PULLUP);  
  pinMode(streetLedPin, OUTPUT);  
  
  barrier.attach(27);  
  barrier.write(0);  
  
  scale.begin(loadCellDT, loadCellSCK);  
  scale.set_scale();  
  scale.tare();  
  
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);  
  display.setTextColor(SSD1306_WHITE);  
  
  WiFi.begin(ssid, password);  
}
```

Void loop:

```
void loop() {  
  unsigned long now = millis();  
  
  trafficLight(now);  
  readLoadCell(now);  
  streetLight(now);  
  updateOLED(now);  
  sendToNodeRed(now);  
}
```

Traffic Light Sequence Function:

```
void trafficLight(unsigned long now) {

    if (digitalRead(irSensor) == LOW) {
        pedestrianWaiting = true;
        pedestrianReported = true;
    }

    switch (currentState) {

        case RED:
            digitalWrite(ledRed, HIGH);
            digitalWrite(ledYellow, LOW);
            digitalWrite(ledGreen, LOW);

            redTime = pedestrianWaiting ? 8000 : 5000;

            if (pedestrianWaiting) {
                barrier.write(90);
                barrierUp = true;
            } else {
                barrier.write(0);
                barrierUp = false;
            }

            if (now - stateMillis >= redTime) {
                stateMillis = now;
                pedestrianWaiting = false;
                barrier.write(0);
                barrierUp = false;
                currentState = GREEN;
            }
            break;

        case GREEN:
            digitalWrite(ledRed, LOW);
            digitalWrite(ledYellow, LOW);
            digitalWrite(ledGreen, HIGH);

            greenTime = vehicleDetected ? 8000 : 5000;

            if (now - stateMillis >= greenTime) {
                stateMillis = now;
                blinkMillis = now;
                blinkCount = 0;
                blinkState = false;
                currentState = GREEN_BLINK;
            }
            break;
    }
}
```

```

case GREEN_BLINK:
    if (now - blinkMillis >= 200) {
        blinkMillis = now;
        blinkState = !blinkState;
        digitalWrite(ledGreen, blinkState);
        blinkCount++;
    }

    if (blinkCount >= 10) {
        digitalWrite(ledGreen, LOW);
        stateMillis = now;
        currentState = YELLOW;
    }
    break;

case YELLOW:
    digitalWrite(ledRed, LOW);
    digitalWrite(ledYellow, HIGH);
    digitalWrite(ledGreen, LOW);

    if (now - stateMillis >= 3000) {
        stateMillis = now;
        currentState = RED;
    }
    break;
}
}

```

Load Cell Function:

```

void readLoadCell(unsigned long now) {
    if (now - loadMillis < 500) return;
    loadMillis = now;

    if (scale.is_ready()) {
        vehicleDetected = scale.get_units(1) >=
weightThreshold;
    }
}

```

Street Light Function:

```

void streetLight(unsigned long now) {
    if (now - ldrMillis < 200) return;
    ldrMillis = now;

    int ldr = analogRead(ldrPin);
}

```

OLED Function:

```
void updateOLED(unsigned long now) {
    if (now - oledMillis < 200) return;
    oledMillis = now;

    long total = 0;
    long remain = 0;

    display.clearDisplay();
    display.setTextSize(1);
    display.setCursor(0,0);

    if (currentState == RED) {
        display.print("RED");
        total = redTime;
    } else if (currentState == GREEN) {
        display.print("GREEN");
        display.setCursor(60,0);
        display.print("GO!!!");
        total = greenTime;
    } else if (currentState == GREEN_BLINK) {
        display.print("GREEN");
        display.setCursor(60,0);
        display.print("WAIT");
        total = 2000;
    } else {
        display.print("YELLOW");
        display.setCursor(60,0);
        display.print("SLOW");
        display.setCursor(60,8);
        display.print("DOWN");
        total = 3000;
    }

    remain = total - (now - stateMillis);
    if (remain < 0) remain = 0;

    display.setTextSize(2);
    display.setCursor(50,16);
    display.print(remain / 1000);

    // Loading bar
    display.drawRect(14, 42, 100, 6, SSD1306_WHITE);
    int fill = map(remain, 0, total, 0, 100);
    display.fillRect(14, 42, fill, 6, SSD1306_WHITE);

    display.setTextSize(1);
    display.setCursor(0, 52);
    display.print("Street:");
    display.print(streetBrightness);

    display.display();
}
```

Node-RED Function & Setup:

```
void sendToNodeRed(unsigned long now) {
  if (now - sendMillis < 1000) return;
  sendMillis = now;

  if (WiFi.status() != WL_CONNECTED) return;

  HTTPClient http;
  http.setTimeout(300);

  long countdown = 0;
  if (currentState == RED) countdown = redTime - (now - stateMillis);
  else if (currentState == GREEN) countdown = greenTime - (now - stateMillis);
  else if (currentState == YELLOW) countdown = 3000 - (now - stateMillis);
  else countdown = 2000 - (now - blinkMillis);
  if (countdown < 0) countdown = 0;

  String url = String(serverUrl) +
    "?traffic=" +
    (currentState==RED?"RED":currentState==GREEN?"GREEN":currentState==YELLOW?"YELLOW":"
    GREEN_BLINK") +
    "&pedestrian=" + String(pedestrianReported) +
    "&vehicle=" + String(vehicleDetected) +
    "&street=" + String(streetBrightness) +
    "&barrier=" + String(barrierUp) +
    "&countdown=" + String(countdown/1000);

  http.begin(url);
  http.GET();
  http.end();

  pedestrianReported = false;
}
```

5.0 RESULT OF PROTOTYPING

5.1 Hardware



Figure 5.1.1: Front View Prototype

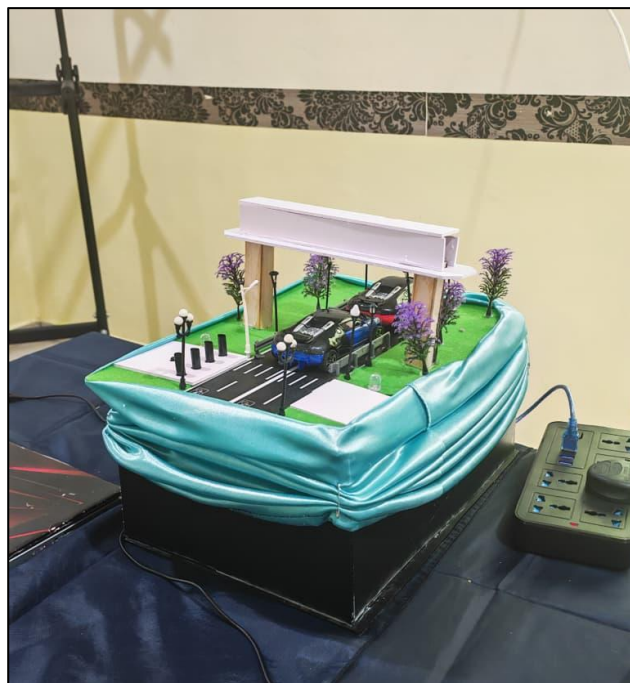
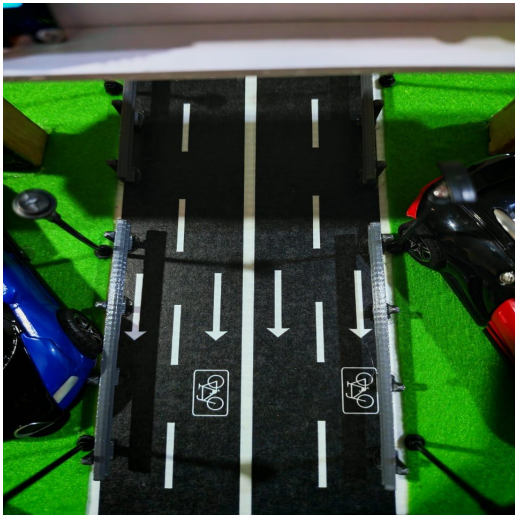
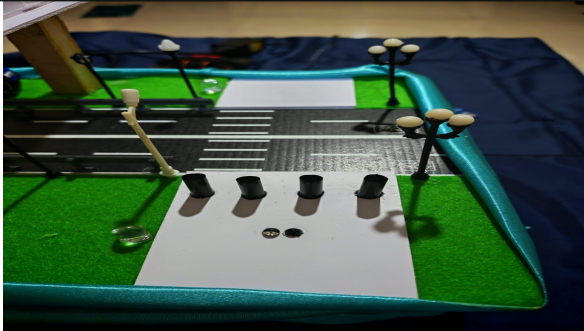
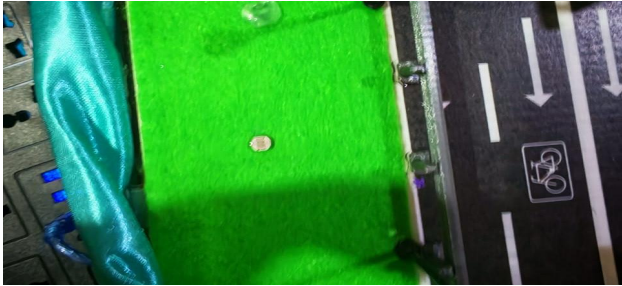


Figure 5.1.2: Back View Prototype

Table 5.1.1: Hardware Prototype

Traffic Light & OLED Display	
Load Cell Sensor	

IR Sensor and Servo Motor (Barrier)		
LDR		
Street Light (PWM)	0	
	60	
	255	

5.2 Software

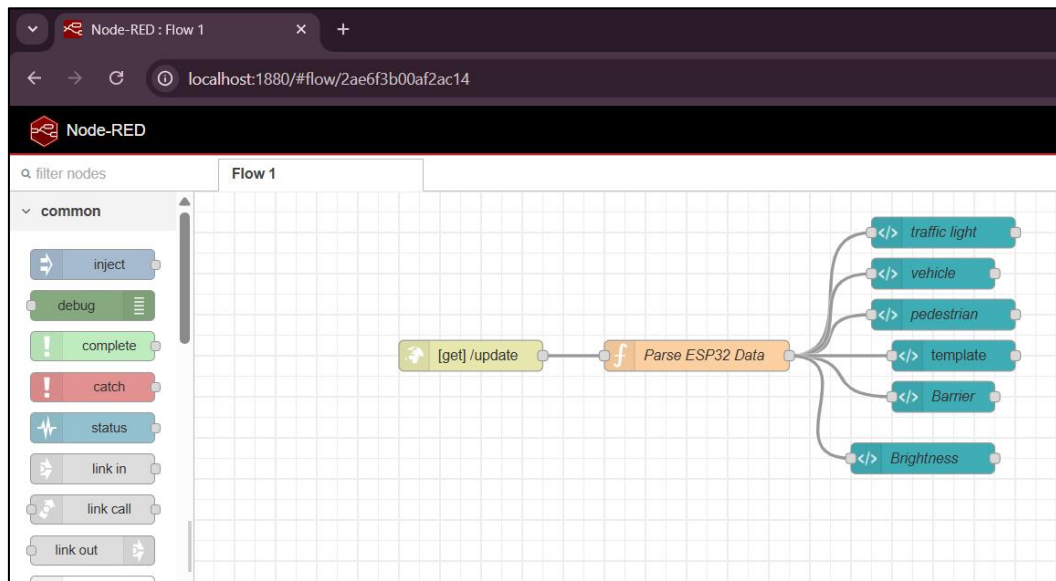


Figure 5.2.1: Node-RED Flow

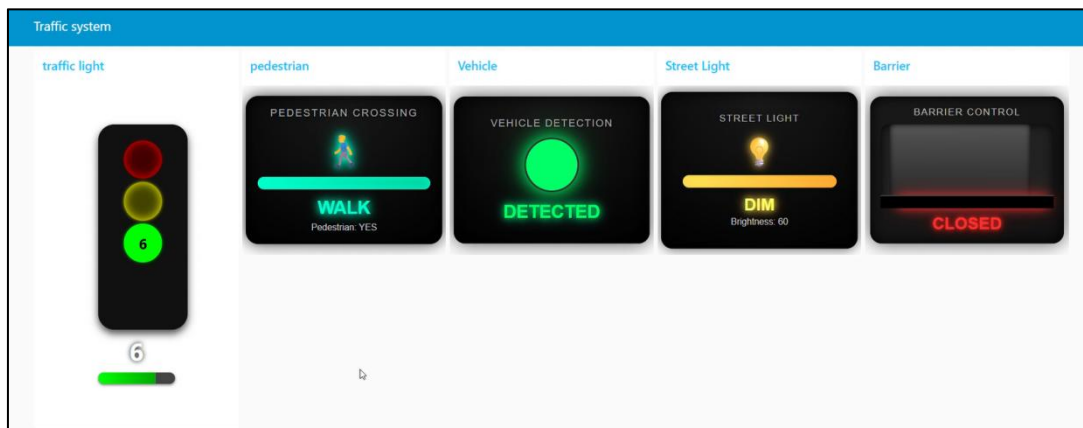
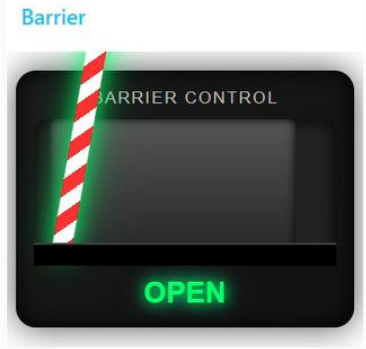
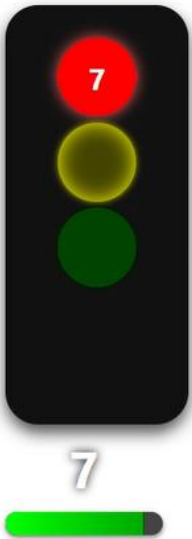


Figure 5.2.2: Node-RED Dashboard/ui

Table 5.2.1: Node-RED Dashboard

PEDESTRIAN	When pedestrian detected	pedestrian  PEDESTRIAN CROSSING WALK Pedestrian: YES
	When pedestrian not detected	pedestrian  PEDESTRIAN CROSSING STOP Pedestrian: NO
VEHICLE	When Vehicle detected	Vehicle  VEHICLE DETECTION DETECTED
	When vehicle not detected	Vehicle  VEHICLE DETECTION NO VEHICLE

BARRIER	When barrier is open	 Barrier control panel showing 'OPEN' status. The panel is black with a red and white striped barrier icon on the left. The text 'BARRIER CONTROL' is at the top, and 'OPEN' is displayed in green at the bottom.
	When barrier is closed	 Barrier control panel showing 'CLOSED' status. The panel is black with a red and white striped barrier icon on the left. The text 'BARRIER CONTROL' is at the top, and 'CLOSED' is displayed in red at the bottom.
STREET LIGHT	When sunny street light = 0	 Street light control panel showing 'DIM' status. The panel is black with a light bulb icon at the top. The text 'STREET LIGHT' is at the top, and 'DIM' is displayed in yellow at the bottom. Below 'DIM' is 'Brightness: 0'.
	When cloudy street light = 60	 Street light control panel showing 'DIM' status. The panel is black with a light bulb icon at the top. The text 'STREET LIGHT' is at the top, and 'DIM' is displayed in yellow at the bottom. Below 'DIM' is 'Brightness: 60'.
	When dark street light = 255	 Street light control panel showing 'BRIGHT' status. The panel is black with a light bulb icon at the top. The text 'STREET LIGHT' is at the top, and 'BRIGHT' is displayed in yellow at the bottom. Below 'BRIGHT' is 'Brightness: 255'.

TRAFFIC LIGHT	Traffic Light Green : 7second: when vehicle detected 5second: When vehicle not detected	traffic light 
	Traffic Light Red: 7second: when pedestrian detected. 5second: when pedestrian not detected	traffic light 

6.0 COSTS OF THE PROJECT

Component		Quantity	Unit Cost (RM)	Total Cost (RM)
ESP32 Board	Development	1.00	33.00	33.00
ESP32 Extend Board		1.00	5.00	5.00
OLED Display		1.00	15.00	15.00
led		3.00	0.60	1.80
Servo Motor		1.00	12.00	12.00
Street Light (PWM)		1 set	6.00	6.00
Buzzer		1.00	2.60	2.60
Load Cell sensor		1.00	8.00	8.00
HX711		1.00	5.00	5.00
IR Sensor		1.00	2.00	2.00
LDR		1.00	0.20	0.20
Battery Holder		1.00	1.20	1.20
Material (PVC board)	Foam	3.00	4.00	12.00
Total Cost				103.80

7.0 CONCLUSION

In conclusion, the traffic light prototype using the ESP32 microcontroller was successfully designed and implemented. The system was able to control the Red, Yellow, and Green LEDs according to a predefined timing sequence, effectively simulating the basic operation of a real traffic light system.

This project achieved its main objectives by demonstrating the use of **ESP32 GPIO control**, **timing-based programming**, and **basic electronic circuit design**. The inclusion of pedestrian crossing concepts, although not physically implemented, helped to enhance understanding of how traffic and pedestrian movements can be managed safely in real-world applications.

Overall, this project provided valuable hands-on experience in embedded system development and improved problem-solving and programming skills. In the future, the system can be further enhanced by adding **pedestrian push buttons**, **countdown displays**, **audible alarms**, or **sensor-based control**, making it more suitable for smart traffic and smart city applications.

8.0 Link for presentation

<https://youtu.be/Y-ykNpIn4L4?si=qM72-3gEGN7D6eW0>